Research Project Dissertation
ISIMA
M2 - ICS

Efficient Tool Management in Flexible Manufacturing: A Branch and Price Approach to the Job Grouping Problem

Présenté par :
Felipe SHOEI OTIAI

Tutors :
Rafael Colares
Annegret Wagler

Campus des Cézeaux - 1 rue de la Chebarde, TSA 01225, 63178 Aubière

Abstract

Flexible Manufacturing Systems (FMS) play a crucial role in modern industry by enabling rapid adaptation to market changes and product diversification. Efficient tool management is essential in FMS, particularly when the total number of required tools exceeds the machine's tool magazine capacity, leading to time-consuming tool switching operations.

This work focuses on the Tool Switching Instants Problem (TSIP), also known as the Job Grouping Problem, which seeks to minimize the number of complete tool configuration changes in a machine tool magazine. This problem arises in various industrial contexts, including printed circuit board production, as well as in computational settings analogous to the k-server problem. We consider the case of a single machine with uniform tool sizes.

Our approach builds on recent column generation (CG) formulation for the Job Grouping Problem. Specifically, we propose: (i) an analysis of the pricing algorithm for the Job Grouping Problem; (ii) a study of the linear relaxation of the formulation; and (iii) the development of a branch-and-price procedure based on this approach.

This integrated strategy aims to provide efficient and practical solutions for tool management while enhancing the operational flexibility of manufacturing systems.

Keywords : Job grouping, tool management, column generation.

Contents

1 Introduction

A Flexible Manufacturing System (FMS) is crucial in modern industry because it enables manufacturers to respond rapidly to market changes and product diversification by efficiently adjusting tools and machine setups. It allows different parts to be processed on the same machines with fewer costs and fewer modifications, thanks to the ability to quickly change tools and operations. However, when the total number of tools required by all part types exceeds the capacity of the machine's tool magazine, tool loadings or switchings become unavoidable. These switching operations consume time and can delay production, reducing some of the flexibility that FMS aims to provide.

In this context, efficient tool management becomes crucial [Crama, 1997]. Two primary objectives are typically considered: minimizing the number of tool switches and minimizing the number of tool switching instants.

Minimizing the number of tool switches is commonly referred to as the Job Sequencing and Tool Switching Problem (SSP). This problem is particularly important when the time required to change a tool is significant relative to the processing times of the parts [Crama et al., 1994].

Minimizing the number of tool switching instants (TSIP), also known as the Job Grouping Problem, is another relevant objective in production environments where machines must be stopped during tool changes [Konak et al., 2008]. In this context, the tool magazine can be exchanged at specific instants, allowing multiple tool switches to occur simultaneously. In this setting, jobs are grouped so that each group can be processed using a set of tools that fits within the tool magazine capacity.

In this work, we focus on the Uniform TSIP, considering a single machine and uniform tool sizes. Our research builds on a Column Generation (CG) approach, leveraging the CG formulation proposed by [Crama et al., 1994]. Specifically, we aim to (i) analyze the pricing problem arising from the CG formulation, (ii) propose a Branch-and-Price procedure using the Ryan–Foster branching rule, and (iii) compare the strength of the resulting relaxation and its computational performance with the exact formulation proposed by [Jans and Desrosiers, 2013].

2 Job Grouping Problem

2.1 Context

The Job Grouping Problem arises in several industrial contexts where machines must process different jobs while operating under limited tool capacity constraints. A common example is the production of printed circuit boards (PCBs) [Tzur, 2004]. In these environments, automated machines must load a specific set of component feeders to assemble each PCB type, but the machine can only accommodate a limited number of feeders at a time. Consequently, feeders must be replaced when switching from one PCB type to another, a process that is both time-consuming and disruptive. In this context, feeders play a role analogous to tools, and grouping jobs that require similar feeders can significantly reduce setup operations and improve production efficiency.

Another related application appears in the k-server problem, as described in [Privault, 2000]. In this setting, servers correspond to tools and memory locations play a role analogous to a machine's tool magazine capacity. The k-server problem models situations such as memory allocation, where a limited number of servers must handle requests arriving over time. In this context, requests are equivalent to jobs, tools correspond to files, and the cache memory plays a role analogous to the tool magazine.

2.2 Problem Description

The Tool Switching Instant Problem (TSIP), also known as the Job Grouping Problem, can be described as follows. Let J be a set of jobs, with $|J| = n$, that must be processed on a machine. A set of tools T, with $|T| = m$, is available to process these jobs. Each job $j_i \in J$ requires a subset of tools $T_i \subseteq T$ in order to be executed.

The available tools are stored in a repository (also called a *magazine*) with capacity b, which can hold at most b tools at any time. At any moment, the subset of tools present in the repository defines a *configuration* $C \subseteq T$ such that $|C| \leq b$. For simplicity, we assume that the repository is always filled to capacity, i.e., $|C| = b$.

A job $j_i \in J$ can be processed under a configuration C if all the tools required by the job are present in that configuration, that is, if $T_i \subseteq C$. We also assume that the magazine has sufficient capacity for all jobs, so that $|T_i| \leq b$ for every $i \in J$.

The objective of the TSIP is to partition the set of jobs into groups such that all jobs in the same group can be processed using a common configuration, while minimizing the number of configurations required. Each configuration corresponds to a *tool switching instant*, during which the machine is stopped and the set of tools in the magazine is replaced.

Example.

Consider a small instance of the Job Grouping Problem with tool magazine capacity $b = 3$. Let the set of tools be $T = \{A, B, C, D, E\}$ and the set of jobs be $J = \{1, 2, 3, 4, 5\}$.

Each job requires a subset of tools $T_i \subseteq T$ as shown in Table 1 and in the Table 2 where a value 1 indicates that job j_i requires tool t..

Job j_i	Required tools T_i
1	A,B
2	A,C
3	C,D
4	D
5	D,F

Table 1: Tool requirements for each job

	Jobs				
Tools	j_1	j_2	j_3	j_4	j_5
A	1	1	0	0	0
B	1	0	0	0	0
C	0	1	1	0	0
D	0	0	1	1	1
F	0	0	0	0	1

Table 2: Tool–job incidence matrix

Since the magazine capacity is $b = 3$, each configuration may contain at most three tools. The minimal number of configurations required for this instance is 2. And an optimal job grouping is shown in Table 3 and Table 4.

Configuration	Tool Set	Jobs processed
C_1	A,B,C	1,2
C_2	C,D,F	3, 4, 5

Table 3: Feasible grouping and configurations

	C_1		C_2		
Tools	j_1	j_2	j_3	j_4	j_5
A	1	1	0	0	0
B	1	0	0	0	0
C	0	1	1	0	0
D	0	0	1	1	1
F	0	0	0	0	1

Table 4: Incidence matrix with jobs grouped by configuration.

2.3 Exact Formulations

[Tang, 1988b] observe that the Job Grouping Problem can be viewed as a generalization of the classical bin packing problem, and it is NP-hard when the magazine capacity satisfies $b \geq 2$ [Konak et al., 2008]. This computational complexity motivates the development of exact optimization approaches capable of solving moderate-size instances. [Adjiashvili et al., 2015] presented a polynomial algorithm that minimizes the number of switching instants for a fixed sequence of jobs, a problem closely related to the Tool Switching Problem and the associated tooling problem [Tang, 1988a].

Exact approaches have been explored in the literature, although they are relatively few. [Tang, 1988b] proposed a branch-and-bound procedure to find an optimal solution.

[Crama and Oerlemans, 1994] proposed a column generation approach. They related the problem to a set covering formulation with an exponential number of variables, and therefore used a column generation method to solve it. This formulation constitutes the basis of the method studied in this work and will be presented in detail in Section **??**

[Denizel, 2003] uses a formulation called Bin Packing with Sharing and applies a Lagrangian decomposition to propose a branch-and-bound procedure, which outperforms the method proposed by Tang.

[Jans and Desrosiers, 2013] proposed an alternative integer programming formulation that reduces symmetry by introducing variable reduction and lexicographic ordering constraints. Their asymmetric representatives formulation significantly strengthens the linear relaxation and improves computational performance. This formulation will be described in the next Section 2.3.1.

2.3.1 Asymmetric Representatives Formulation (ARF)

The asymmetric representatives formulation (ARF) proposed by [Jans and Desrosiers, 2013] is based on a formulation originally introduced for clustering problems to eliminate symmetry among identical clusters. In the context of the Job Grouping Problem, symmetry arises because configurations are indistinguishable: any permutation of configurations produces an equivalent solution. This symmetry often leads to weak linear relaxations and large branch-and-bound trees.

The key idea of the ARF is to represent each group of jobs by its *lowest indexed job*. This job acts as the *representative* of the group. Consequently, each potential group corresponds to a unique job index, which removes the symmetry between identical configurations.

Let $J = \{1, \ldots, n\}$ be the set of jobs and T the set of tools. For each job $i \in J$, let $T_i \subseteq T$ denote the set of tools required to process job i. The magazine capacity is denoted by b.

The ARF introduces the following binary decision variables:

$$v_{ih} = \begin{cases} 1 & \text{if job } i \text{ is assigned to the group represented by job } h \\ 0 & \text{otherwise} \end{cases} \qquad \forall i, h \in J,\ i \geq h$$

$$y_{th} = \begin{cases} 1 & \text{if tool } t \text{ is loaded in the configuration represented by job } h \\ 0 & \text{otherwise} \end{cases} \qquad \forall t \in T,\ h \in J$$

If $v_{hh} = 1$, then job h is the representative of a group and therefore a configuration is activated.

The ARF formulation can be written as follows:

$$\min \quad \sum_{h \in J} v_{hh} \tag{1}$$

$$\text{s.t.} \quad \sum_{h=1}^{i} v_{ih} = 1 \qquad \forall i \in J \tag{2}$$

$$v_{ih} \leq v_{hh} \qquad \forall h \in J,\ i = h, \ldots, n \tag{3}$$

$$v_{ih} \leq y_{th} \qquad \forall h \in J,\ i = h, \ldots, n,\ t \in T_i \tag{4}$$

$$\sum_{t \in T} y_{th} \leq b\, v_{hh} \qquad \forall h \in J \tag{5}$$

$$v_{ih} \in \{0, 1\},\ y_{th} \in \{0, 1\} \tag{6}$$

The objective function minimizes the number of active groups, which corresponds to minimizing the number of tool switching instants. Constraints (2) ensure that each job is assigned to exactly one group. Since a group is identified by its lowest indexed job, job i can only be assigned to groups with representative $h \leq i$. Constraints (3) guarantee that jobs can only be assigned to active groups. Constraints (4) ensure that if job i is assigned to group h, all tools required by job i must be present in the corresponding configuration. Constraints (5) enforce the magazine capacity limit.

An important practical aspect highlighted by [Jans and Desrosiers, 2013] is that the ordering of the jobs can significantly influence the strength of the linear relaxation. In particular, ordering the jobs according to the number of required tools from *high to low* produces stronger relaxations and better computational performance for the ARF formulation. Their computational study shows that this ordering provides the best average lower bounds and overall performance.

For this reason, the High-to-Low ordering is adopted in our implementation and computational experiments. The ARF formulation is also used as a benchmark model for evaluating both the linear relaxation and the computational performance of the proposed exact approach.

Nevertheless, [Jans and Desrosiers, 2013] report that for larger instances other formulations based on lexicographic ordering constraints may achieve better overall computational performance.

3 Column Generation Approach

In order to solve the Job Grouping Problem, [Crama and Oerlemans, 1994] proposed an exact method based on a column generation framework. The approach relies on a set covering formulation in which each column corresponds to a feasible configuration of tools capable of processing a subset of jobs. Since the number of possible configurations is exponential in the number of tools, it is not practical to enumerate all of them explicitly.

To overcome this difficulty, the authors apply a column generation procedure in which a restricted master problem is solved iteratively while new configurations are generated

dynamically through a pricing subproblem.

As illustrated in Figure 1, the column generation process alternates between solving the restricted master problem and the pricing subproblem.

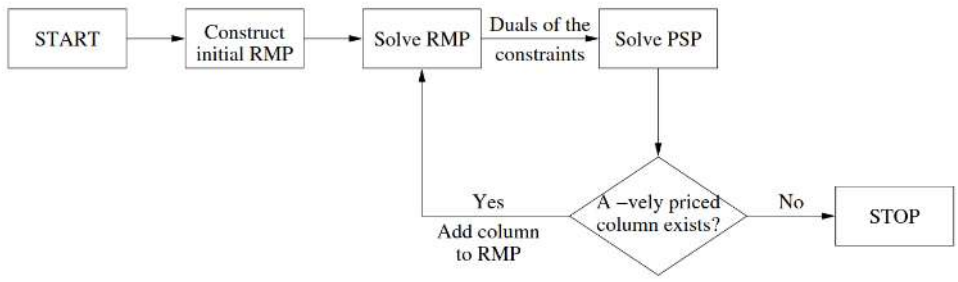

Figure 1: General structure of a column generation framework [Muter et al., 2010].

3.1 Set Covering Interpretation

A natural way to formulate the Job Grouping Problem is through a set covering perspective. In this formulation, each feasible configuration of tools corresponds to a set capable of processing a subset of jobs.

Let

$$J(C) = \{\, j \in J \mid T_j \subseteq C \,\}$$

be the set of jobs that configuration C can process.

A subset $\mathcal{C}^\star \subset \mathcal{C}$ is said to be a *cover* of J if

$$\bigcup_{C \in \mathcal{C}^\star} J(C) = J.$$

The objective is therefore to determine a cover of minimum cardinality:

$$\min_{\mathcal{C}^\star \subseteq \mathcal{C}} \{\, |\mathcal{C}^\star| \;:\; \bigcup_{C \in \mathcal{C}^\star} J(C) = J \,\}.$$

In other words, we seek the smallest set of configurations capable of processing all jobs. Since each configuration represents a feasible set of tools loaded in the magazine, minimizing the number of configurations corresponds to minimizing the number of job groups.

Formally, let $\mathcal{C}$ denote the set of all feasible configurations such that $C \subseteq T$ and $|C| = b$. A configuration C covers job $j_i \in J$ if $T_i \subseteq C$. For each configuration C, we define

$$J(C) = \{\, j_i \in J \mid T_i \subseteq C \,\}.$$

The problem therefore consists of selecting a minimum-size subset $\mathcal{C}^* \subseteq \mathcal{C}$ such that

$$\bigcup_{C \in \mathcal{C}^\star} J(C) = J.$$

Because the number of feasible configurations grows exponentially with the number of tools, [Crama and Oerlemans, 1994] propose solving this formulation using a column generation approach.

3.2 Restricted Master Problem

Let $\mathcal{C}' \subseteq \mathcal{C}$ denote the current set of configurations considered in the restricted master problem.

For each configuration $C \in \mathcal{C}'$, we introduce a variable $x_C \geq 0$ indicating whether configuration C is selected. In the linear relaxation these variables are continuous.

The Restricted Master Problem (RMP) is formulated as follows:

$$\min \quad \sum_{C \in \mathcal{C}'} x_C \tag{7}$$

$$\text{s.t.} \quad \sum_{C \in \mathcal{C}' : T_i \subseteq C} x_C = 1 \qquad \forall i \in J \tag{8}$$

$$0 \leq x_C \leq 1 \qquad \forall C \in \mathcal{C}'. \tag{9}$$

The objective (7) minimizes the number of configurations used. Constraint (8) ensures that every job is covered by one selected configuration. Since the set of feasible configurations $\mathcal{C}$ is extremely large, the master problem is initially solved using only a restricted subset $\mathcal{C}'$ of configurations.

New configurations are then generated through a pricing problem.

3.3 Pricing Subproblem

After solving the restricted master problem, we obtain dual variables π_i associated with the covering constraints.

A new column (configuration) should be generated whenever it has negative reduced cost. The reduced cost of configuration C is

$$\text{rc}(C) = 1 - \sum_{i : T_i \subseteq C} \pi_i.$$

Thus, the pricing problem consists of solving

$$\max_{C \subseteq T,\, |C| = b} \sum_{i : T_i \subseteq C} \pi_i.$$

If

$$\max_{C : |C| = b} \sum_{i : T_i \subseteq C} \pi_i > 1,$$

then a configuration with negative reduced cost exists and must be added to the restricted master problem.

3.3.1 Integer Programming Formulation of the Pricing Problem

The pricing problem can also be formulated as an integer program.

Let z_t be a binary variable equal to 1 if tool t is selected in the configuration. Let y_i be a binary variable equal to 1 if job i is covered by the configuration.

$$\max \quad \sum_{i \in J} \pi_i y_i \tag{10}$$

$$\text{s.t.} \quad y_i \leq z_t \qquad \forall i \in J,\ \forall t \in T_i \tag{11}$$

$$\sum_{t \in T} z_t = b \tag{12}$$

$$z_t \in \{0,1\} \qquad \forall t \in T \tag{13}$$

$$y_i \in \{0,1\} \qquad \forall i \in J \tag{14}$$

Constraint (11) ensures that job i can only be covered if all required tools are selected in the configuration, while constraint (12) enforces the tool magazine capacity.

3.3.2 Analysis of the Pricing Procedure

The pricing problem described in the previous section plays a central role in the efficiency of the column generation framework. Given the dual variables π_i obtained from the Restricted Master Problem, the goal of the pricing problem is to identify a configuration of tools that maximizes the total dual value of the jobs that can be processed by that configuration.

From a computational perspective, this problem is challenging. As discussed by [Crama and Oerlemans, 1994]. This problem it is known to be NP-hard. Even when $\pi_i = 1$ for all i in jobs, the problem remains NP-hard [Gallo et al., 1980].

In the column generation algorithm proposed by [Crama and Oerlemans, 1994], the pricing problem is solved using a binary tree enumeration procedure. In this approach, the search space of feasible configurations is explored through a branching process in which tools are progressively included or excluded from the candidate configuration. Each node of the enumeration tree represents a partial configuration of tools, and bounding rules based on the remaining capacity of the tool magazine are used to prune subtrees that cannot yield improving columns.

The effectiveness of this procedure depends heavily on the pruning rules. In practice, the size of the search tree is influenced by the similarity between jobs and by the capacity of the tool magazine, since these factors determine how early infeasible or non-improving configurations can be discarded. However, in the worst case the enumeration tree may explore all subsets of tools of size up to b. In such a case, the tree has maximum depth $|T|$ and a branching factor of two, leading to a worst-case complexity on the order of

$$O(2^{|T|}),$$

which is exponential in the number of tools. Consequently, the pricing procedure may become computationally expensive for instances with a large number of tools.

From a modern perspective, the pricing problem can be interpreted as a particular case of the *Set Union Knapsack Problem* (SUKP). In this problem, each item corresponds to a job with an associated profit, while selecting an item requires including all the tools associated with that job. The objective is to maximize the total profit of the selected jobs while respecting a capacity constraint on the number of tools. The SUKP is a well-known NP-hard problem and several heuristic and approximation approaches have been proposed in the literature.

Goldschmidt et al. [1994] formally define the problem and show that it remains NP-hard even under very restrictive conditions, including instances where items contain only two elements and all weights and profits are equal. The authors also propose an exact dynamic programming algorithm for solving the SUKP. However, the algorithm has exponential worst-case complexity, with running time proportional to

$$O\left(m\,n\,b\sum_{j=1}^{n}2^{|h(j)|}\right),$$

where n is the number of elements (tools in the job grouping problem), m is the number of items (jobs in our case), b is the knapsack capacity (corresponding to the magazine capacity), and $h(j)$ denotes the set of elements adjacent to element j in the item-adjacency graph.

This complexity shows that the algorithm becomes exponential in the size of the interaction structure between items, although it may run in polynomial time for special cases where the adjacency structure is limited.

Deliberately, [Crama and Oerlemans, 1994] avoid solving the pricing problem to optimality at each iteration. As discussed by the authors, solving the pricing formulation exactly would typically generate only a small number of columns per iteration of the column generation procedure. This could lead to a large number of iterations of the master problem.

Furthermore, the computational environment used in their experiments imposed additional limitations. In particular, the LP solver LINDO used by [Crama and Oerlemans, 1994] did not support dynamic column addition or reoptimization. Consequently, each time a new column was generated the entire linear program had to be re-solved from scratch, which was computationally expensive. For this reason, the authors preferred to generate multiple columns per iteration using a tree enumeration procedure instead of solving the pricing problem exactly.

Modern optimization solvers, however, provide features that significantly reduce this overhead. In particular, solvers such as SCIP support LP reoptimization, allowing the master problem to be updated and re-solved efficiently after new columns are added. This capability makes it possible to reconsider alternative pricing strategies, including exact mixed-integer formulations, without incurring the same computational penalties observed in earlier implementations.

For these reasons, we conduct a computational analysis of the pricing procedure. More precisely, we compare the binary tree enumeration strategy described in [Crama and Oerlemans, 1994] with an exact mixed-integer programming formulation of the pricing problem implemented using the SCIP optimization framework. This analysis aims to evaluate the practical efficiency of the enumeration-based method and to assess whether modern integer programming solvers can provide a competitive alternative for solving the pricing problem.

3.4 Proposed Framework

The complete algorithm proposed by [Crama and Oerlemans, 1994] follows a column generation scheme combined with constructive heuristics and bounding procedures. The main steps of the method are summarized in Algorithm 1, and presented in this section.

Algorithm 1 Column Generation Framework for the Job Grouping Problem

Require: Set of jobs J, set of tools T, tool magazine capacity C

```
Generate initial feasible groups using seven sequential heuristics (Section 3.4.2)
Compute lower bounds LB_SW and LB_MSW (Section 3.4.1)
if LB = UB then
    STOP
end if
Build the initial Restricted Master Problem (RMP)
Run Nemhauser and Weasley RMP heuristic (Section 3.4.4)
Dietrich columns generation procedure (Section 3.4.3)
Solve the RMP
while  do
    Extract dual variables π_i associated with job-covering constraints
    Solve the pricing problem using binary tree search on reduced costs (Section 3.3)
    if pricing problem is completely explored then
        Compute the Farley lower bound LB_Farley (Section ??)
    end if
    if no column with negative reduced cost is generated then
        BREAK
    end if
end while
Run Nemhauser and Weasley RMP heuristic (Section 3.4.4)
```

The algorithm begins by generating an initial set of feasible configurations using several constructive heuristics. These configurations define the initial columns of the restricted master problem. Lower bounds are also computed before the column generation procedure begins.

At each iteration, the restricted master problem is solved and the dual variables are used to define the pricing problem. If a configuration with negative reduced cost is found, it is added to the master problem and the process continues. Otherwise, the column generation procedure terminates.

Finally, heuristic procedures based on the RMP solution are used to produce feasible integer solutions.

3.4.1 Lower Bounds

This section presents some known lower bounds from the literature that are used in this framework. Mainly two bounds are considered:

- the Modified Sweeping lower bound LB_{MSW},
- the Farley lower bound LB_{Farley}.

The Farley lower bound LB_{Farley} is a generic lower bound arising from a column generation framework. This bound requires an optimal solution of the pricing problem in order to scale the dual variables and obtain a valid lower bound, as described in [Desrosiers et al., 2024].

The Modified Sweeping procedure proposed in [Tang, 1988b] provides a valid lower bound for the Job Grouping Problem and is described in the next section.

Tool Capacity Lower Bound (LB_{TC})

Let $|T|$ denote the total number of tools and let C be the tool magazine capacity. A simple lower bound on the number of groups follows from the magazine capacity constraint:

$$LB_{TC} = \left\lceil \frac{|T|}{C} \right\rceil .$$

Since each feasible group can contain at most C tools, at least $\lceil |T|/C \rceil$ groups are required.

Example

Let $C = 2$ and $|T| = 3$, with tools $T = \{1, 2, 3\}$.

Consider two jobs with tool requirements

$$T_1 = \{1, 2\}, \qquad T_2 = \{2, 3\}.$$

No group can include all three tools simultaneously;

$$\left\lceil \frac{3}{2} \right\rceil = 2$$

hence at least 2 groups are necessary.

Independent Set Lower Bound (LB_{IS})

An additional lower bound can be derived by constructing a compatibility graph $G = (J, E)$ where each vertex represents a job.

An edge $(i, j) \in E$ exists if jobs i and j can be processed in the same configuration, that is,

$$|T_i \cup T_j| \leq C.$$

Any independent set in G corresponds to a set of jobs that cannot be grouped together pairwise. Therefore, each job in such a set must belong to a different group, and the size of a maximum independent set provides a valid lower bound for the problem. The construction of this graph is illustrated in Figure 2.

Since computing a maximum independent set is NP-hard, a greedy sweeping procedure is used to approximate it.

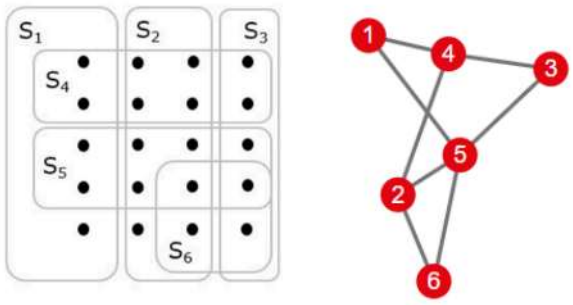

Figure 2: Representation of the job grouping problem and the corresponding compatibility graph.

Algorithm 2 Sweeping Procedure

```
G ← ∅
U ← J
k ← 1
while U ≠ ∅ do
    Select a job in U with the fewest compatible jobs
    Add the job to G
    Remove the job and all its neighbors from U
    k ← k + 1
end while
return |G|
```

Modified Sweeping Lower Bound (LB_{MSW})

The Modified Sweeping lower bound combines the ideas of the tool capacity bound and the independent set bound by recomputing residual tool requirements at each iteration of the sweeping procedure.

At each iteration k of the sweeping algorithm, let G_k be the set of jobs selected so far and let U_k be the set of remaining jobs in the graph. Denote by $\bigcup_{i \in U_k} T_i$ the set of tools required by the remaining jobs.

A valid lower bound at iteration k is then given by

$$|G_k| + \left\lceil \frac{\left|\bigcup_{i \in U_k} T_i\right|}{C} \right\rceil .$$

The maximum value obtained over all iterations defines the Modified Sweeping lower bound LB_{MSW}.

3.4.2 Sequential Heuristics for the Job Grouping Problem

An initial set of feasible job groups and configurations is generated using seven constructive heuristics. Each heuristic sequentially builds maximal feasible groups subject to the tool magazine capacity constraint.

All heuristics follow the same general structure. First, a job requiring the largest number of tools is selected to initialize a group. Then, the corresponding selection rule is applied to greedily choose additional jobs to be included in the group. When no further job can be added without violating the magazine capacity constraint, the group is closed and the procedure is repeated to generate the next group.

Let S denote the current group and let $i \notin S$ be a candidate job. We define:

- t_i : number of tools required by job i;
- b_i^S : number of tools required by both job i and group S;
- p_i^S : number of jobs not in S that can be processed if job i is included in S.

Maximal Intersection, Minimal Union, and MIMU Heuristics

The Maximal Intersection (MI) heuristic selects the job that maximizes b_i^S, breaking ties arbitrarily.

The Minimal Union (MU) heuristic selects the job that minimizes t_i, breaking ties arbitrarily.

The Maximal Intersection Minimal Union (MIMU) heuristic selects the job that maximizes b_i^S and, in case of ties, chooses the job that minimizes t_i.

Whitney and Gaul Heuristic

The Whitney and Gaul heuristic selects the job $i \notin S$ that maximizes the score

$$\frac{b_i^S + 1}{t_i + 1}.$$

Marginal Gain Heuristic

The Marginal Gain heuristic selects the job that maximizes p_i^S.

Rajagopalan Heuristic

In the Rajagopalan heuristic, weights are assigned to tools as follows. For each tool k, let

$$w_k = \text{number of unassigned jobs that require tool } k.$$

The weight of a job i is then defined as

$$r_i = \sum_{k \in T_i \setminus S} w_k,$$

where T_i denotes the set of tools required by job i. The job with the maximum value of r_i is selected. The first job of each group is also chosen according to this criterion.

Modified Rajagopalan Heuristic

The Modified Rajagopalan heuristic differs from the original version by redefining the tool weights. For each tool k, let

$$w'_k = \text{number of jobs already assigned to } S \text{ that require tool } k.$$

The job selection rule then follows the same weighted principle as in the original Rajagopalan heuristic.

3.4.3 Dietrich Column Generation Procedure

Multiple initial columns are generated using this procedure.

For each job $i \in J$, a group is initialized with job i as the seed. Additional jobs are then added sequentially according to the rule that maximizes

$$\frac{p_i^S}{t_i - b_i^S},$$

where S denotes the current group.

The authors do not explicitly specify a rule for starting a new group when the current group becomes full, that is, when no additional job can be added without violating the magazine capacity constraint.

At the end of the procedure, a collection of $|J|$ feasible job groupings is obtained. These groupings are used to generate an initial pool of columns for the restricted master problem.

3.4.4 Nemhauser-Wolsey RMP Heuristic

The Nemhauser–Wolsey heuristic constructs a feasible integer solution from the Restricted Master Problem (RMP). Configurations (columns) are selected greedily: at each step, the configuration covering the largest number of currently uncovered jobs is chosen until all jobs are covered.

4 Branch-and-Price

The column generation framework proposed by [Crama and Oerlemans, 1994] provides strong lower and upper bounds for the Job Grouping Problem through a set covering formulation. In particular, the authors show how the optimal value of the linear relaxation can be computed using a column generation procedure, despite the fact that the pricing subproblem is NP-hard.

Their computational experiments demonstrate that the lower bounds obtained through column generation are very tight. Among the 1130 randomly generated instances tested, the lower bound matched the optimal solution for 1119 instances. As a result, the proposed method was able to solve almost all instances to optimality in practice.

However, the framework does not constitute a fully exact algorithm. The method relies on the comparison between upper and lower bounds and does not incorporate a branching mechanism capable of guaranteeing optimality in all cases. Consequently, when the bounds do not coincide, the procedure cannot certify optimality.

This limitation arises because the column generation framework is used only to solve the linear relaxation of the set covering formulation. To obtain a provably optimal solution, the column generation procedure must be embedded within a branching scheme, resulting in a *branch-and-price* algorithm.

Branch-and-price combines column generation with branch-and-bound by dynamically generating variables during the search process. At each node of the branching tree, a restricted master problem is solved and new columns are generated through the pricing subproblem whenever necessary. This integration allows the algorithm to explore the integer solution space while maintaining the advantages of the column generation approach.

In this work, we extend the column generation framework of [Crama and Oerlemans, 1994] by embedding it into a branch-and-price procedure. This enables the algorithm to certify optimality and provides a fully exact method for solving the Job Grouping Problem.

4.1 Branch-and-Price Framework

To obtain an exact method, the global lower bound provided by the linear relaxation must coincide with the value of a known feasible solution, which serves as a global upper bound. When this situation does not occur, branching arises naturally in order to further explore the space of integer solutions.

Branching partitions the set of feasible integer solutions into smaller subproblems by introducing additional constraints that eliminate the current fractional solution. Each node of the resulting search tree therefore represents a restricted feasible domain of the original problem.

By recursively exploring these nodes, the algorithm may either improve the global lower bound or identify better feasible integer solutions. If the global lower bound obtained through the exploration of the branching tree matches the value of a known feasible solution, the optimality gap is closed and optimality can be certified.

Embedding column generation within such a branching framework leads to the *branch-and-price* method. In this approach, the linear relaxation associated with each node of the search tree is solved using column generation rather than by explicitly enumerating

all variables. This allows the method to handle formulations with a very large number of potential variables while still benefiting from the bounding mechanism of branch-and-bound. The general structure of the branch-and-price framework is illustrated in Figure 3.

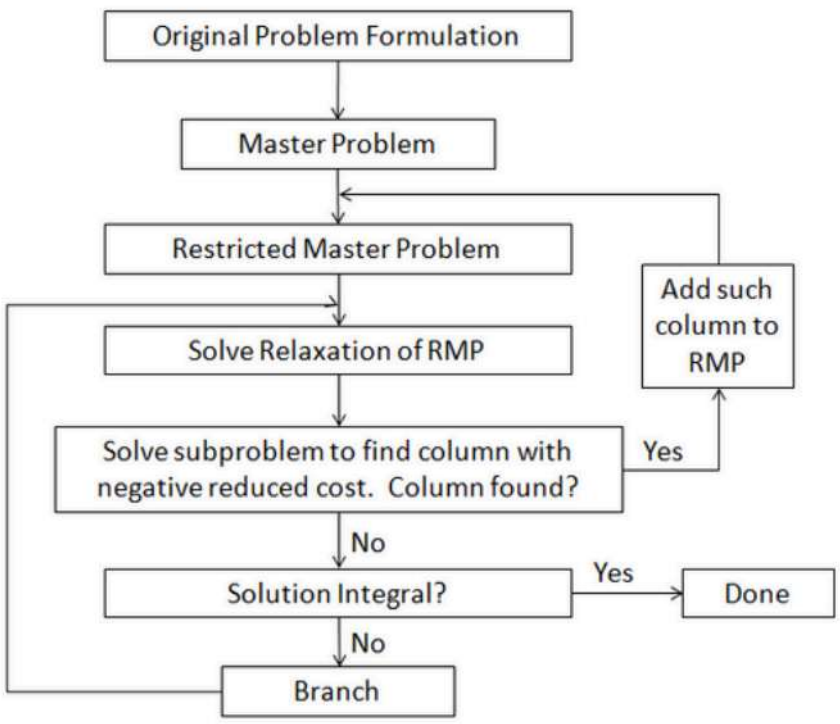

Figure 3: Branch-and-Price framework. Adapted from [Akella et al., 2010].

The integration of branching with column generation is not straightforward. Branching decisions introduce additional restrictions that may affect the structure of the pricing problem responsible for generating new columns. In particular, branching constraints may modify the reduced cost calculation and therefore alter the structure of the pricing subproblem. Consequently, branching rules must be designed carefully so that the pricing problem remains tractable and compatible with the column generation framework.

Furthermore, solving nodes in branch-and-price algorithms is typically more computationally expensive than in classical branch-and-bound procedures, since each node requires solving a column generation problem. For this reason, the choice of branching strategies and the quality of the bounds obtained during the search play a crucial role in the efficiency of the algorithm [Desrosiers et al., 2024].

A good branching rule should ideally lead to an improvement of either the primal bound or the dual bound at the created nodes. In practice, effective branching decisions aim to produce balanced search trees in which both child nodes significantly restrict the feasible region and strengthen the bounds [Desrosiers et al., 2024].

On the other hand, the classical branching rule used in branch-and-bound does not appear to be effective in the context of column generation [Uchoa et al., 2024]. In our case, classical branching would impose $x_c = 1$ or $x_c = 0$ for a fractional x_c^* variable.

As discussed by [Uchoa et al., 2024], branching on individual generated column variables can be ineffective in formulations exhibiting strong symmetry, since many alternative

column combinations may represent the same solution in the original variable space. Fixing $x_c = 0$ only removes a single configuration from the feasible region, which typically has little impact since the pricing problem can generate many alternative configurations representing similar solutions. In contrast, enforcing $x_c = 1$ may overly restrict the feasible region by forcing a specific configuration to be part of the solution.

For these reasons, branching rules used in branch-and-price algorithms are typically designed to impose structural restrictions on the elements composing the columns rather than on individual columns themselves.

Several branching rules have been proposed in the literature [Desrosiers et al., 2024]. One of the most commonly used rules in set partitioning and set covering formulations is the Ryan–Foster branching rule [Ryan and Foster, 1981]. This rule has been successfully applied in a variety of column generation based algorithms, including problems such as Crew Scheduling, Vehicle Routing, Bin Packing, and Graph Coloring.

For these reasons, and due to the relatively simple modifications required in the pricing problem, we adopt the Ryan–Foster branching rule in this work. The details of this rule are described in the next section.

4.2 Ryan–Foster Branching Rule

The Ryan–Foster branching rule was originally introduced for set partitioning formulations in crew scheduling problems [Ryan and Foster, 1981] and later became one of the most widely used branching strategies in branch-and-price algorithms [Desrosiers et al., 2024]. The rule avoids branching directly on column variables and instead imposes structural restrictions on the relationship between pairs of elements.

Let I denote the set of items and let $\mathcal{P}$ denote the set of columns, where each column represents a feasible subset of items. For a column $p \in \mathcal{P}$, let

$$a_{ip} = \begin{cases} 1 & \text{if item } i \text{ belongs to column } p, \\ 0 & \text{otherwise.} \end{cases}$$

Given an optimal fractional solution of the restricted master problem, the rule identifies a pair of items (i, j) that appear together in some columns but not in others. In such a situation, the algorithm creates two child nodes that impose opposite relationships between these two items.

This branching scheme is often referred to as *same and different branching*.

Same branch In the first branch, items i and j are forced to appear in the same column. This means that any feasible column must either contain both items or none of them. Formally, the following condition must hold for all generated columns:

$$a_{ip} = a_{jp} \qquad \forall p \in \mathcal{P}.$$

This restriction ensures that whenever item i is included in a column, item j must also be included.

Different branch In the second branch, items i and j are forbidden from appearing together in the same column. Therefore, columns containing both items are not allowed. This can be expressed as

$$a_{ip} + a_{jp} \leq 1 \qquad \forall p \in \mathcal{P}.$$

As a consequence, the two items must be assigned to different columns in any feasible integer solution.

Job Grouping case In our case, due to the structure of the problem, it is natural to consider jobs as items. Accordingly, let $\mathcal{C}$ denote the set of all feasible configurations (columns). For a configuration $c \in \mathcal{C}$, let

$$a_{jc} = \begin{cases} 1 & \text{if configuration } c \text{ processes job } j, \\ 0 & \text{otherwise.} \end{cases}$$

4.2.1 Choosing Fractional Pairs

A key property underlying the Ryan–Foster rule is that fractional solutions of the linear relaxation necessarily induce fractional relationships between some pairs of items. Let x_c^* denote the value of configuration c in the current solution of the restricted master problem. For any pair of jobs (i, j), consider the aggregated quantity

$$\sum_{c \in \mathcal{C}} a_{ic} a_{jc} x_c^*,$$

which represents the sum of x_c^* for all configurations c that process job i and j together.

If the current solution is integral, that is $x_c^* \in \{0, 1\}$ for all $c \in \mathcal{C}$, then all such sums are necessarily integer values, since they correspond to counting whether the selected configuration processes the pair (i, j) together.

However, when the solution of the master problem is fractional, there must exist at least one pair of jobs (i, j) for which this sum is fractional.

To identify such a fractional pair, we compute a triangular matrix M_{ij} for $i > j$, defined as

$$M_{ij} = \sum_{c \in \mathcal{C}} a_{ic} a_{jc} x_c^*.$$

The branching pair is then selected among the fractional values of M_{ij}, typically choosing the pair whose value is closest to 0.5, that is, the most fractional one.

For a selected pair (i, j), two child nodes are created: one enforcing that jobs i and j must be processed by the same configuration, and another enforcing that jobs i and j must be processed by different configurations.

4.2.2 Modified Pricing Problem

A key advantage of the Ryan–Foster rule is that it preserves the structure of the pricing subproblem. Instead of fixing individual column variables, the rule modifies the feasibility conditions used when generating new columns. The pricing problem is therefore

solved under additional constraints ensuring that newly generated configurations satisfy the imposed relationship between the selected pair of jobs.

According to the branching rule imposed at a child node, the pricing problem can be modified as follows. If jobs i and j must be processed by the same configuration, they can be replaced by a single artificial job representing the merged pair (i, j). This implicitly guarantees that both jobs are always processed together. This modification is applied as a preprocessing step before solving the pricing problem at each node.

For different rule in child nodes, the restriction must be explicitly enforced in the pricing problem. Let $D \subseteq J \times J$ denote the set of pairs of jobs (i, j) that cannot be processed in the same configuration. In the exact integer pricing formulation (10)–(12), the following constraint can be added:

$$y_i + y_j \leq 1 \qquad \forall (i, j) \in D.$$

which ensures that no generated configuration processes both jobs i and j simultaneously. In the binary search tree used to solve the pricing problem, this constraint can be implemented as a feasibility check that prunes branches violating the condition.

4.3 Illustrative Example of Ryan–Foster Branching

Consider a small instance with five jobs

$$J = \{J_1, J_2, J_3, J_4, J_5\}.$$

Assume the column generation procedure finished at root node and has generated the following configurations.

	Jobs				
Configuration	J_1	J_2	J_3	J_4	J_5
c_1	1	1	0	0	0
c_2	0	1	1	0	0
c_3	0	0	1	1	0
c_4	0	0	0	1	1
c_5	1	0	0	0	1

Table 5: Configuration matrix (a_{jc}).

Each row represents a feasible configuration, and each column corresponds to a job. A value $a_{jc} = 1$ indicates that job j is processed in configuration c.

Fractional RMP Solution

Suppose the Restricted Master Problem produces the fractional solution

$$x_{c_1} = 0.5, \quad x_{c_2} = 0.5, \quad x_{c_3} = 0.5, \quad x_{c_4} = 0.5, \quad x_{c_5} = 0.5.$$

The covering constraints are satisfied:

$$
\begin{aligned}
J_1: &\quad x_{c_1} + x_{c_5} = 0.5 + 0.5 = 1,\\
J_2: &\quad x_{c_1} + x_{c_2} = 0.5 + 0.5 = 1,\\
J_3: &\quad x_{c_2} + x_{c_3} = 0.5 + 0.5 = 1,\\
J_4: &\quad x_{c_3} + x_{c_4} = 0.5 + 0.5 = 1,\\
J_5: &\quad x_{c_4} + x_{c_5} = 0.5 + 0.5 = 1.
\end{aligned}
$$

Thus the RMP solution is feasible but fractional.
The objective value of the LP relaxation is

$$z_{LP} = \sum_{c \in \mathcal{C}} \lambda_c = 0.5 \times 5 = 2.5.$$

Computing the Pair Matrix

Following the Ryan–Foster rule, we compute the matrix M_{ij}. For the fractional solution we obtain

$$
\begin{aligned}
M_{12} &= 0.5\\
M_{23} &= 0.5\\
M_{34} &= 0.5\\
M_{45} &= 0.5\\
M_{15} &= 0.5
\end{aligned}
$$

Several pairs have fractional values. Since these values are equal, the branching pair can be selected arbitrarily. Suppose the pair (J_1, J_2) is chosen.

Branching

Two child nodes are created:

- **SAME**: jobs J_1 and J_2 must appear in the same configuration.
- **DIFFER**: jobs J_1 and J_2 cannot appear together.

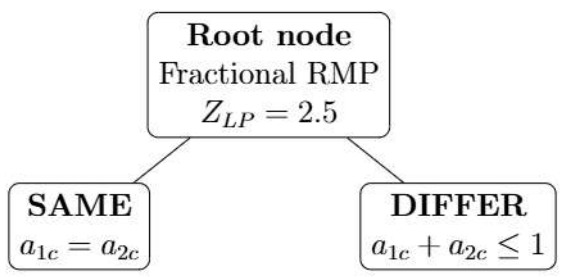

Figure 4: Ryan–Foster branching on pair (J_1, J_2).

Impact on the Pricing Problem

When applying Ryan–Foster branching on the pair (J_1, J_2), the pricing problem must be modified to respect the branching decision.

SAME branch. In this branch, jobs J_1 and J_2 must always appear together in the same configuration. Therefore, any configuration containing only one of the two jobs must be prohibited. In the restricted master problem (RMP), this implies fixing the variables corresponding to configurations that contain only one of these jobs to zero (e.g., $x_{c_2} = 0$ and $x_{c_5} = 0$).

In the pricing problem, jobs J_1 and J_2 are merged into a single *super-job* J_{12} whose tool requirements correspond to the union of the tools required by J_1 and J_2. This guarantees that newly generated configurations either include both jobs or exclude them both.

DIFFER branch. In this branch, jobs J_1 and J_2 are not allowed to appear together in the same configuration. Therefore, any configuration containing both jobs must be prohibited. In the RMP, this implies fixing the corresponding variable to zero (e.g., $x_{c_1} = 0$).

$$y_1 + y_2 \leq 1.$$

The overall branching framework is illustrated in Figure 4.3. Each node of the tree has its own pricing problem and RMP, they are modified according to the branching decisions taken along the path from the root node. The columns (configurations) generated at a node are passed to its child nodes. However, when moving to a child node, variables corresponding to configurations that violate the branching constraints must be fixed to zero in the restricted master problem (RMP).

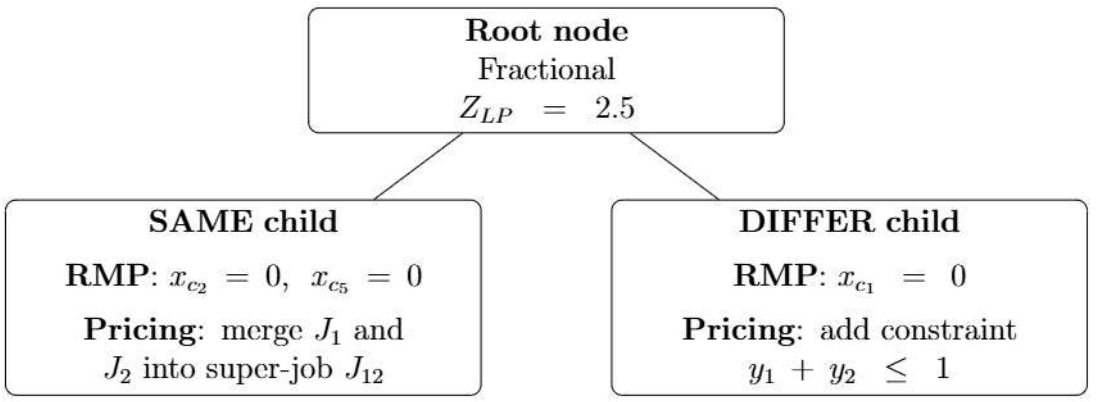

Figure 5: Ryan–Foster branching on pair (J_1, J_2).

5 Implementation Details

The Branch-and-Price algorithm was implemented in C++ using the C++17 standard. The implementation relies on the SCIP Optimization Suite (version 10.0.0) [Hojny et al., 2025] for solving the Restricted Master Problem and managing the branch-and-price tree.

The implementation of the Branch-and-Price scheme follows the standard plug-in interface described in the SCIP documentation [Hojny et al., 2025], the technical guide

ISIMA Branch-Cut-and-Price was used as a practical support for understanding and structuring the implementation [Toussaint, 2013].

5.1 SCIP as a Branch-and-Price Framework

SCIP (Solving Constraint Integer Programs) is a highly flexible optimization framework that supports branch-and-bound, cutting planes, and column generation within a unified architecture [Hojny et al., 2025]. Its design is based on a plug-in system

This modular architecture makes SCIP particularly well-suited for Branch-and-Price algorithms, where three non-standard components must be provided by the user alongside the base model:

- **Pricer**: generates new columns (configurations) with negative reduced cost whenever the current restricted master problem (RMP) is solved to optimality.
- **Branching rule**: decides how to branch when the RMP relaxation yields a fractional solution, in a way that is compatible with the column generation pricing procedure.
- **Constraint handler**: stores, transforms, propagates, and enforces the branching decisions (SAME / DIFFER constraints) locally along each node of the search tree.

Beyond plug-in support, SCIP provides LP reoptimization via warm starting at each iteration of the column generation algorithm, as discussed in Section 3.3.2.

5.2 Master Problem

The class `RunOptimizationScip` acts as the central orchestrator of the solver. It owns the main SCIP instance, constructs and maintains the Restricted Master Problem (RMP), and coordinates the registration of all plug-ins.

Constructor and initialisation. The constructor receives an instance of the class `InstanceTosp`, which reads and parses a problem instance file, storing the job–tool incidence matrix and the tool magazine capacity b. Inside the constructor, SCIP is created and initialised, the minimisation sense is set, and presolving is disabled to preserve the dual variables required by the column generation procedure.

Column generation framework. Before the column generation loop begins, some actions are performed to generate initial configurations, compute a lower bound, and obtain an upper bound via heuristics.

- `generateHeuristicConfigs()` — generates an initial set of feasible configurations using the seven greedy heuristics described in Section 3.4.2.
- `generateDietrichConfigs()` — generates additional configurations using the Dietrich construction described in Section 3.4.3.
- `solveSetCoverHeuristic()` — applies the Nemhauser–Wolsey greedy heuristic over the RMP to produce a feasible integer solution, as described in Section 3.4.4.

- `getLowerBoundSweepingProcedure()` — computes the Modified Sweeping lower bound LB_{MSW} described in Section 3.4.1, which serves as a reference point for evaluating the quality of the LP relaxation.

The method `solveExact()` constructs the exact set-covering model, adds the job-covering constraints, and registers the three custom plug-ins with SCIP: the pricer `ObjPricerTosp`, the constraint handler `ConshdlrSamediff`, and the branching rule `BranchruleRF_Jobs` that are described in the next sections.

5.3 Pricer

The two pricer classes inherit from `ObjPricer`, the base pricer class provided by SCIP:

- `ObjPricerTosp` — implements the binary-tree enumeration pricing strategy.
- `Pricer_BranchAndBound` — implements the exact IP pricing strategy.

The functions below are the main standard callbacks defined by the `ObjPricer` interface that are implemented by any custom pricer.

- `scip_init` Initialisation callback of the plug-in, called once after SCIP creates the transformed problem. It translates all original constraint and variable pointers stored in the master into their internal SCIP transformed pointers.
- `scip_redcost` Called by SCIP whenever the LP relaxation at the current node has been solved to optimality. It delegates to `pricing()` the routine described below.

The core column-generation routine is implemented by `pricing()`. It coordinates the following pipeline on each call:

1. **Read dual values.** For each job j, the dual variable π_j is read from the corresponding covering constraint.
2. **Read branching constraints.** `readBranchingConstraints()` extracts all active SAME and DIFFER constraints along the current node's path in the branch-and-price tree, as described in Section 4.2.
3. **Build artificial jobs.** `buildSameComponents()` groups jobs into components connected by SAME constraints, and `buildArtificialJobs()` collapses each component into a single artificial super-job whose tool set is the union of its members' tools and whose dual value is the sum of their duals, as described in Section 4.2.2.
4. **Build conflict graph.** `buildArtificialDiffGraph()` translates all active DIFFER constraints into a conflict adjacency list over the artificial jobs. During the pricing search, whenever an artificial job is selected, all its neighbours in this graph are immediately prohibited.

5. **Solve the pricing subproblem.** In `class ObjPricerTosp`, `binarySearch()` enumerates candidate configurations in non-increasing order of dual values using a recursive binary tree.

   In `class Pricer_BranchAndBound`, `solveExactPricing()` solves the IP formulation (10)–(12) using an internal SCIP instance.

6. **Add columns.** Every improving configuration found is added to the master problem via `addColumnToMaster()`.

5.4 Branching Rule

The branching rule `class BranchruleRF_Jobs` inherits from `ObjBranchrule` and implements the Ryan–Foster rule described in Section 4.2.

The main default plug-in callback that is overridden is `scip_execlp`, invoked by SCIP whenever the LP relaxation at the current node is fractional and the pricer has confirmed that no improving column exists.

The procedure proceeds as follows:

1. **Retrieve fractoional variables.** `SCIPgetLPBranchCands` returns all variables with fractional LP values in the current solution.

2. **Compute pair weights.** Using all fractional variables x_c^*, it computes the pair weight matrix M_{ij} explained in Section 4.2.1.

3. **Select the most fractional pair.** The pair (i, j) whose weight M_{ij} is closest to 0.5 is selected.

4. **Create two child nodes.** Two child nodes are created via `SCIPcreateChild`.

   - **SAME child**: a constraint forcing jobs i and j to appear in the same configuration is created via `SCIPcreateConsSamediffTosp` with argument `same = TRUE` and attached to the child node via `SCIPaddConsNode`.
   - **DIFFER child**: the same procedure is followed with argument `same = FALSE`.

If no fractional pair is found (all $M_{ij} \in \{0, 1\}$), the callback returns `SCIP_DIDNOTRUN`, signalling to SCIP that the current solution is already integral.

This class processes nodes with fractional solutions, identifies pairs on which branching will be performed, and creates child nodes accordingly by adding SAME/DIFFER constraints attached to each child node. These types of constraints, and the way they are enforced, are implemented by a constraint handler described in the next section.

5.5 Constraint Handler

The constraint handler `class ConshdlrSamediff` inherits from `ObjConsHdlr` and is responsible for managing the SAME and DIFFERENT branching constraints along the branch-and-price tree. These constraints are local: they are active only at the node where they were created and at its descendants.

The lifecycle of a branching constraint passes through three stages. First, the branching rule creates the constraint and attaches it to a child node. Second, SCIP transforms the constraint when the node is activated for processing. Third, the constraint is propagated at every LP solve at that node and all its descendants, and is finally deleted when the subtree is pruned. The functions below correspond to each stage of this lifecycle.

Constraint creation: `SCIPcreateConsSamediffTosp`**.** This utility function is called by the branching rule to create a new SAME or DIFFERENT constraint, acting as a constructor for the constraint object. It allocates a data structure (`SCIP_ConsData`) that stores the indices of the two jobs and a Boolean flag `same` indicating the type of restriction. It then calls `SCIPcreateCons`, passing a pointer to `ConshdlrSamediff`, which registers with SCIP which component is responsible for managing the constraint throughout its lifetime.
The items below are the main standard callbacks defined by the `ObjConsHdlr` interface that are implemented by any custom constraint handler.

- `scip_trans` — The transformation callback translates the original constraint pointers into internal SCIP transformed pointers and copies the `SCIP_ConsData` into the transformed problem space. SCIP calls `scip_trans` when a constraint is first activated at a node, signalling that the node is about to be processed.

- `scip_prop` — The propagation callback is the most important method of the constraint handler. It is called by SCIP at every LP solve. It inspects all current SCIP variables: for each variable, it checks whether the variable respect the corresponding restriction encoded in `SCIP_ConsData`. Any variable that violates the constraint is fixed to zero via `SCIPfixVar`. Figure 4.3 illustrates variable fixing at each child node.

- `scip_delete` — The deletion callback frees the `SCIP_ConsData` block when the constraint is released, preventing memory leaks when a subtree is pruned.

5.6 Summary of the Component Interactions

Figure 6 summarises the interactions between the main components during the branch-and-price algorithm.

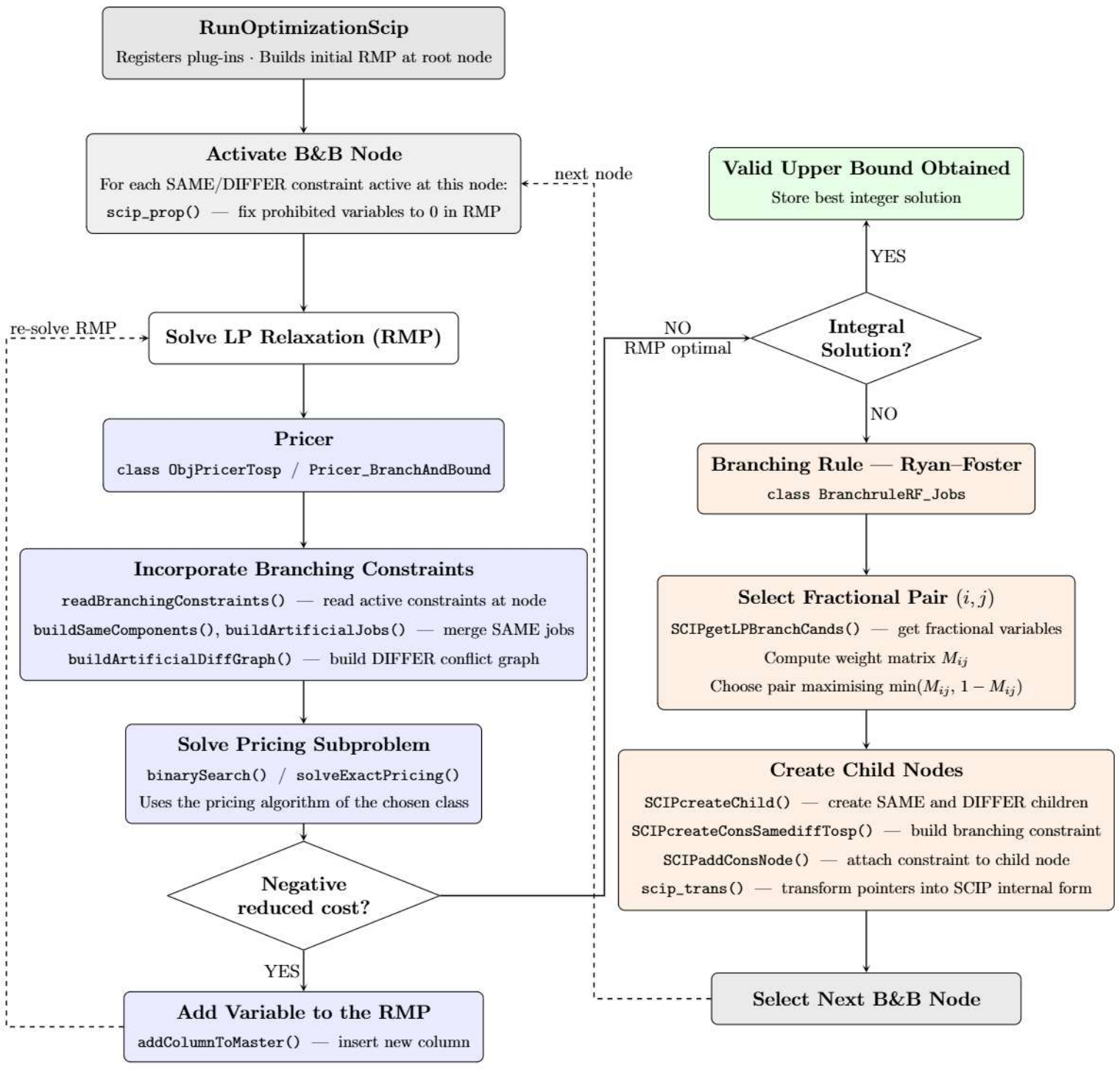

Figure 6: Flowchart of the Branch-and-Price implementation: pricing loop, branching decisions, and constraint propagation at each node of the search tree.

6 Computational Experiments

All experiments were carried out on the `bob` node of the ISIMA cluster, a server equipped with two Intel Xeon E5-2687W v4 processors (12 cores each, 24 physical cores in total) running at 3.00 GHz, a 30 MB cache, and 768 GB of RAM. Each job was submitted via SLURM with a single task and a single CPU core allocated and 4 GB of memory, using the following directives:

```
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=4000
```

The Branch-and-Price framework relies on SCIP version 10.0.0 [Hojny et al., 2025]. And the customized plugins where implemented in C++17 and compiled with GCC 12.2.0 (Debian 12.2.0-14+deb12u1). The Asymmetric Representatives Formulation (ARF) benchmark model was solved using IBM ILOG CPLEX 12.10 [?] via the Concert Technology C++ API.

The computational experiments use instances from the dataset `datD4` provided by Catanzaro et al. [2015], which contains 10 instances. These instances were originally designed for the Tool Switching Problem (SSP); however, since the SSP and the Job Grouping Problem share the same input structure — a job–tool incidence matrix and a tool magazine capacity b — every SSP instance constitutes a valid instance for the Job Grouping Problem.

Instances from Jans and Desrosiers [2013] and Denizel [2003] were requested from the authors, but no response was received before the submission deadline of this work. Therefore, the computational experiments rely on the `datD4` benchmark instances proposed by Catanzaro et al. [2015].

All instances in the `datD4` dataset have identical dimensions, with $|J| = 40$ jobs, $|T| = 60$ tools, and a tool magazine capacity $b = 30$.

6.1 Pricing Problem Evaluation

Table 6 reports the best upper bound obtained by the constructive heuristics together with the optimal value of each instance.

Table 6: Quality of heuristic upper bounds.

Instance	UB	Optimal
D4-1	13	12
D4-2	11	11
D4-3	13	12
D4-4	13	12
D4-5	12	12
D4-6	14	13
D4-7	13	12
D4-8	12	11
D4-9	13	12
D4-10	14	13

6.2 Relaxation Comparison

We compare the strength of different lower bounds, linear relaxations, and heuristic upper bounds for the Job Grouping Problem.

In particular, we consider:

- the modified sweeping bound LB_{MSW} proposed by Tang [1988b],
- the LP relaxation obtained from the column generation formulation of Crama and Oerlemans [1994],
- the LP relaxation of the asymmetric representatives formulation (ARF) proposed by Jans and Desrosiers [2013],
- the best upper bound obtained by the constructive heuristics.

Table 7 summarizes the values obtained for each instance.

Table 7: Comparison of lower bounds, relaxation values, and heuristic upper bounds.

Instance	LB_{MSW}	CG Relaxation	ARF Relaxation	Heuristic UB	Optimal
D4-1	4.23	11.48	3.77	13	12
D4-2	3.00	9.93	3.52	11	11
D4-3	5.20	11.80	3.88	13	12
D4-4	4.00	11.93	3.59	13	12
D4-5	4.00	11.03	3.72	12	12
D4-6	7.00	12.93	3.85	14	13
D4-7	7.00	11.43	3.91	13	12
D4-8	5.40	10.41	3.80	12	11
D4-9	5.00	11.97	3.73	13	12
D4-10	6.00	12.49	3.83	14	13

6.3 Branch-and-Price Performance

We compare the proposed Branch-and-Price algorithm with the ARF formulation solved using SCIP and CPLEX. All experiments were executed with a time limit of 3600 seconds. The table reports the optimal value obtained by the column generation branch-and-price approach (CG), together with the performance of the ARF formulation solved with SCIP and CPLEX. The time limit was set to 3600 seconds.

Table 8: Comparison of exact methods.

	Branch-and-Price (CG)			ARF SCIP		ARF CPLEX	
Instance	Opt	Nodes	Time(s)	UB	Gap(%)	UB	Gap(%)
D4-1	12	1	0.51	14	45.40	13	38.35
D4-2	11	79	20.48	12	49.05	11	38.10
D4-3	12	1	0.56	13	36.80	13	30.77
D4-4	12	5	0.58	14	53.17	13	42.49
D4-5	12	1	0.75	13	49.28	12	40.45
D4-6	13	21	1.31	15	40.78	14	33.12
D4-7	12	1	0.45	13	34.63	13	34.63
D4-8	11	1	0.54	12	40.04	12	33.47
D4-9	12	15	1.80	14	47.26	13	37.17
D4-10	13	1	0.28	14	41.58	14	35.71

6.4 Discussion of Results

The computational results show that the proposed Branch-and-Price (B&P) algorithm consistently outperforms the direct solution of the ARF formulation with general-purpose MIP solvers. For all tested instances of the `D4` dataset, the Branch-and-Price method was able to prove optimality within very short running times, typically below two seconds.

A key observation from the experiments is the strength of the column generation relaxation. Before branching is applied, the column generation procedure already provides lower bounds that are very close to the optimal solution. For most instances, the gap between the column generation relaxation and the optimal integer solution is smaller than one unit.

For example, the column generation relaxation yields values such as 11.48 for instance D4-1 and 11.97 for instance D4-9, while the optimal values are 12 in both cases. Similarly, for instances D4-6 and D4-10 the relaxation values (12.93 and 12.49, respectively) are very close to the optimal solutions (13 in both cases). This confirms the strong bound quality already observed in the work of Crama and Oerlemans [1994].

However, despite the strength of the relaxation, the column generation procedure alone is not sufficient to certify optimality in all cases, since the resulting solution of the restricted master problem may be fractional. In these situations, Branch-and-Price is required to close the remaining optimality gap.

In practice, only a very small number of branching nodes were required to close this gap. Several instances, such as D4-1, D4-3, D4-5, D4-7, D4-8, and D4-10, were solved after exploring a single node in the search tree. Even in the more difficult cases, such as

instance D4-2 where 79 nodes were explored, the overall running time remained below a few seconds.

An important observation is that the column generation relaxation is already extremely tight. For nine out of the ten instances, namely D4-1, D4-3, D4-4, D4-5, D4-6, D4-7, D4-8, D4-9, and D4-10, the difference between the LP relaxation and the optimal solution is smaller than one unit. Only instance D4-2 presents a larger gap. Consequently, the branching phase is mainly required to eliminate the remaining fractional solutions rather than to significantly improve the lower bound.

Another important aspect concerns the quality of the upper bounds provided by the constructive heuristics. In two instances, D4-2 and D4-5, the heuristic solution was already optimal. For the remaining instances, the heuristic solution was slightly above the optimum, typically by one or two units. During the Branch-and-Price search, the algorithm was able to improve these bounds and identify the optimal configuration grouping. In particular, the upper bound was improved for 8 instances, D4-1, D4-3, D4-4, D4-6, D4-7, D4-8, D4-9, and D4-10.

In contrast, the ARF formulation solved with SCIP and CPLEX struggled to close the optimality gap within the imposed time limit of 3600 seconds. Although feasible solutions were found, the remaining gaps were often large, frequently exceeding 30% and reaching more than 50% in some cases.

Overall, these results demonstrate the effectiveness of the Branch-and-Price approach for the Job Grouping Problem. The combination of a very strong column generation relaxation with a limited amount of branching allows the algorithm to prove optimality rapidly, whereas compact MIP formulations remain difficult to solve even with modern state-of-the-art solvers.

7 Conclusion

Flexible Manufacturing Systems require efficient strategies for tool management in order to reduce downtime and improve production flexibility. In this work we studied the Tool Switching Instants Problem (TSIP), also known as the Job Grouping Problem, which aims to minimize the number of tool configuration changes in a machine tool magazine with limited capacity.

Building on the column generation formulation proposed in the literature, this work investigated several aspects of the problem and its solution methods. First, we analysed the structure and complexity of the pricing problem. From a modern perspective, the pricing problem can be interpreted as a particular case of the Set Union Knapsack Problem, which is known to be NP-hard. This observation motivated the study of different approaches for solving the pricing subproblem and highlighted the importance of efficient search strategies.

Second, we studied the strength of the linear relaxation obtained from the column generation formulation. Computational experiments showed that the relaxation provides very strong lower bounds, often within less than one unit of the optimal integer solution. This confirms the effectiveness of the configuration-based formulation for capturing the structure of the problem.

Finally, we developed a branch-and-price algorithm that embeds the column generation procedure within a branching framework. The computational results demonstrate that

this approach is highly effective for the tested instances. In all cases the algorithm was able to prove optimality very quickly, exploring only a small number of branching nodes. The experiments also show that the branch-and-price method significantly outperforms the direct solution of the ARF formulation using general-purpose MIP solvers, which were unable to close the optimality gap within the time limit.

Overall, the results indicate that combining strong column generation relaxations with a branching strategy provides an efficient exact approach for the Job Grouping Problem. The proposed framework successfully exploits the structure of the problem and demonstrates the practical advantages of branch-and-price methods for large-scale combinatorial optimization problems.

Future work may investigate improved pricing strategies, alternative branching rules, and the integration of advanced heuristics to further enhance the performance of the algorithm on larger or more complex instances.

Use of Artificial Intelligence Tools

References

Adjiashvili, D., S. Bosio, and K. Zemmer (2015). Minimizing the number of switch instances on a flexible machine in polynomial time. *Operations Research Letters 43*(3), 317–322.

Akella, M., S. Gupta, and A. Sarkar (2010). Branch and price: Column generation for solving huge integer programs. PDF document. Archived from the original on 2010-08-21.

Catanzaro, D., L. Gouveia, and M. Labbé (2015). Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research 244*(3), 766–777.

Crama, Y. (1997). Combinatorial optimization models for production scheduling in automated manufacturing systems. *European Journal of Operational Research 99*(1), 136–153.

Crama, Y., A. W. J. Kolen, A. G. Oerlemans, and F. C. R. Spieksma (1994, January). Minimizing the number of tool switches on a flexible machine. *Int J Flex Manuf Syst 6*, 33–54.

Crama, Y. and A. G. Oerlemans (1994). A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research 78*(1), 58–80.

Denizel, M. (2003, 04). Minimization of the number of tool magazine setups on automated machines: A lagrangean decomposition approach. *Operations Research 51*, 309–320.

Desrosiers, J., M. E. Lübbecke, G. Desaulniers, and J.-B. Gauthier (2024). Branch-and-price. In *Column Generation and Decomposition Techniques in Integer Programming*. Springer.

Gallo, G., P. Hammer, and B. Simeone (1980). Quadratic knapsack problems. *Mathematical Programming*.

Goldschmidt, O., D. Nehme, and G. Yu (1994). Note on the set-union knapsack problem. *Naval Research Logistics 41*, 833–842.

Hojny, C., M. Besançon, K. Bestuzheva, S. Borst, A. Chmiela, J. Dionísio, L. Eifler, M. Ghannam, A. Gleixner, A. Göß, A. Hoen, R. van der Hulst, D. Kamp, T. Koch, K. Kofler, J. Lentz, S. J. Maher, G. Mexi, E. Mühmer, M. E. Pfetsch, S. Pokutta, F. Serrano, Y. Shinano, M. Turner, S. Vigerske, M. Walter, D. Weninger, and L. Xu (2025, November). The SCIP Optimization Suite 10.0. Technical report, Optimization Online.

Jans, R. and J. Desrosiers (2013). Efficient symmetry breaking formulations for the job grouping problem. *Computers Operations Research 40*(4), 1132–1142.

Konak, A., S. Kulturel-Konak, and M. Azizoğlu (2008). Minimizing the number of tool switching instants in flexible manufacturing systems. *International Journal of Production Economics 116*(2), 298–307.

Muter, I., S. I. Birbil, and K. Bülbül (2010, November). Simultaneous column-and-row generation for large-scale linear programs with column-dependent-rows.

Privault, Caroline, F. G. (2000, 11). K-server problems with bulk requests: An application to tool switching in manufacturing. *Annals of Operations Research - Annals OR 96*, 255–269.

Ryan, D. M. and B. A. Foster (1981). An integer programming approach to scheduling. In A. Wren (Ed.), *Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling*, pp. 269–280. Amsterdam: North-Holland Publishing Company.

Tang, Christopher S., D. E. V. (1988a, October). Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Oper. Res. 36*(5), 767–777.

Tang, Christopher S., D. E. V. (1988b, October). Models arising from a flexible manufacturing machine, part ii: Minimization of the number of switching instants. *Oper. Res. 36*(5), 778–784.

Toussaint, H. (2013, April). Introduction au branch cut and price et au solveur scip (solving constraint integer programs). Technical Report RR-13-07, LIMOS.

Tzur, Michal, A. A. (2004, 02). Minimization of tool switches for a flexible manufacturing machine with slot assignment of different tool sizes. *IIE Transactions 36*, 95–110.

Uchoa, E., A. Pessoa, and L. Moreno (2024, August). *Optimizing with Column Generation: Advanced Branch-Cut-and-Price Algorithms*, Volume 2024 of *Cadernos do LOGIS*. LOGIS – Núcleo de Logística Integrada e Sistemas, Universidade Federal Fluminense. Available at: `https://OptimizingWithColumnGeneration.github.io`.